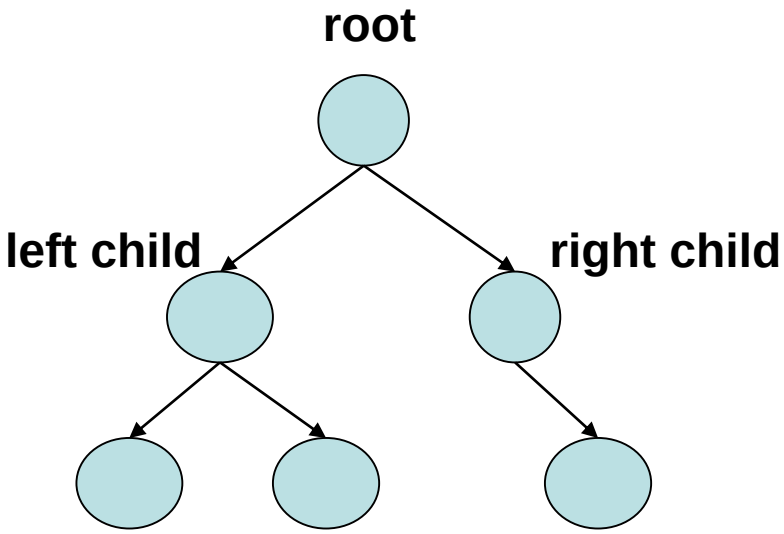
BINARY TREES
&&
TREE TRAVERSALS

DEFINITION : Binary Tree

- A binary tree is made of nodes

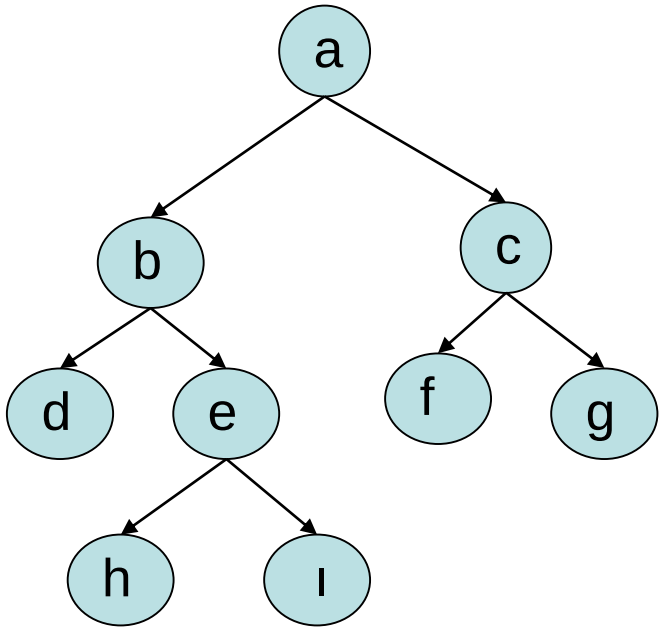


- Each node contains
  - a "left" pointer -- left child
  - a "right" pointer – right child
  - a data element.

- The "root" pointer points to the topmost node in the tree. The left and right pointers recursively point to smaller

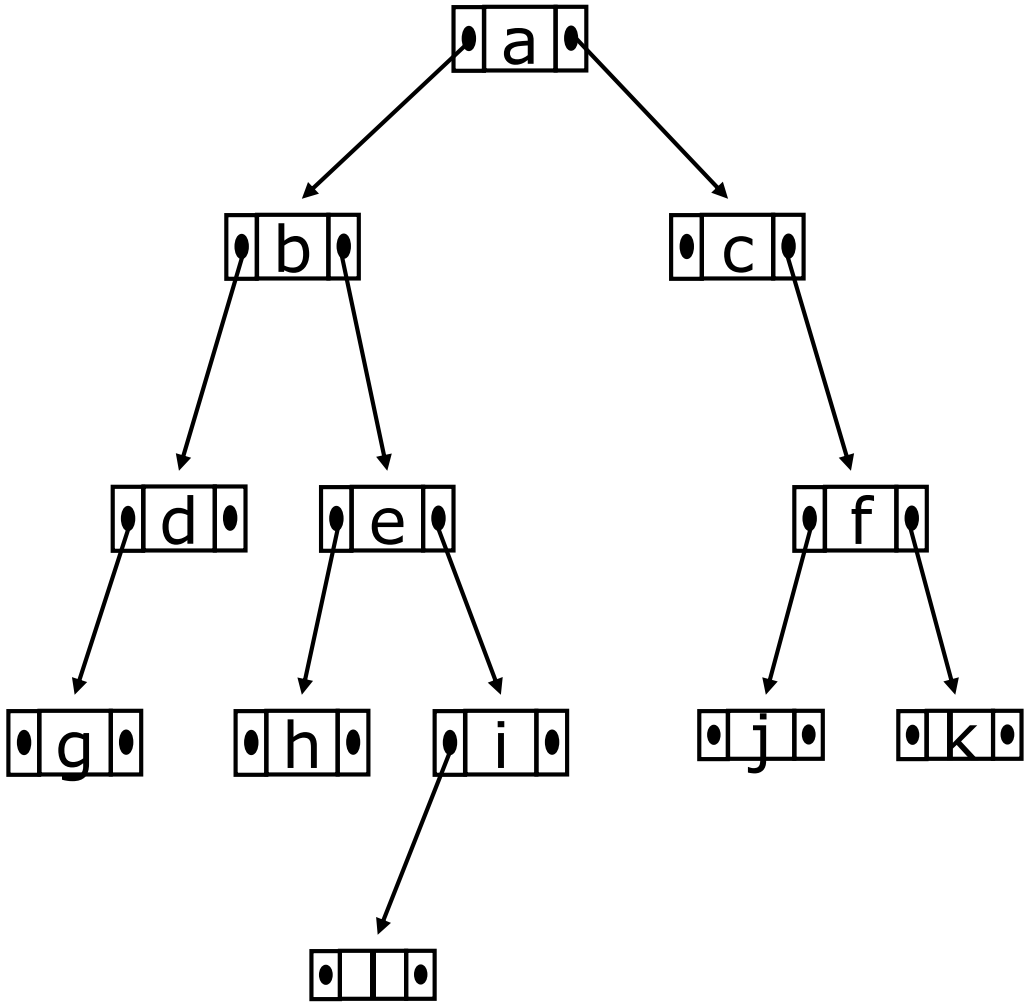
- "subtrees" on either side. A null pointer represents a binary tree with no elements -- the empty tree.

DEFINITION : Binary Tree



- The size of a binary tree is the number of nodes in it
  - This tree has size 9
- The depth of a node is its distance from the root
  - a is at depth zero
  - e is at depth 2
- The depth of a binary tree is the depth of its deepest node
  - This tree has depth 3

DEFINITION : Binary Tree



- The size of tree??
- The depth of tree??

A Typical Binary Tree Declaration

```
struct node {
        int data;
        struct node * left;
        struct node * right;          };
```

Create a binary tree

EXAMPLE:

- Get numbers from user till -1.
- Insert a new node with the given number into the tree in the correct place
- Rule : each right node will be greater than its root and each left node will be less than its root

Create a binary tree : EXAMPLE

```
typedef struct node * BTREE;

/* CREATE A NEW NODE */

 BTREE new_node(int data)
{
        BTREE p;
        p=( BTREE)malloc(sizeof(struct node));

        p->data=data;
        p->left=NULL;
        p->right=NULL;

        return p;
}
```

Create a binary tree : EXAMPLE

```
typedef struct node * BTREE;


/* INSERT DATA TO  TREE */

BTREE insert(BTREE root, int data)
{
    if(root!=NULL)
     {      if(data<root->data)          root->left= insert(root->left,data);
            else                         root->right=insert(root->right,data);
     }

     else {root=new_node(data);}
     return root;

}
```

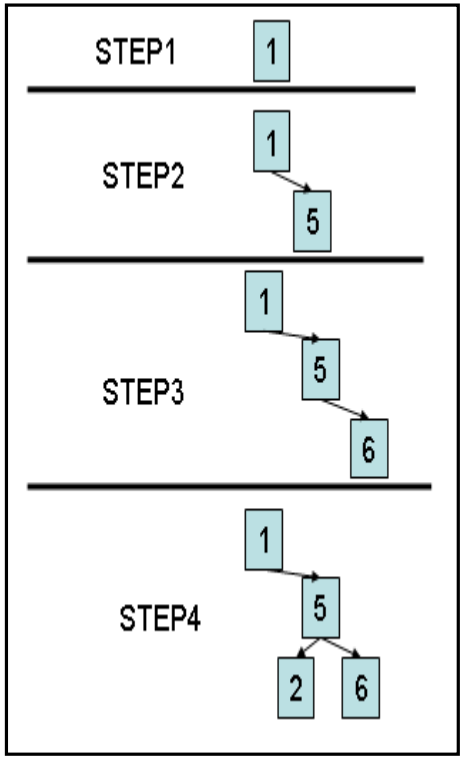
Create binary tree : Example

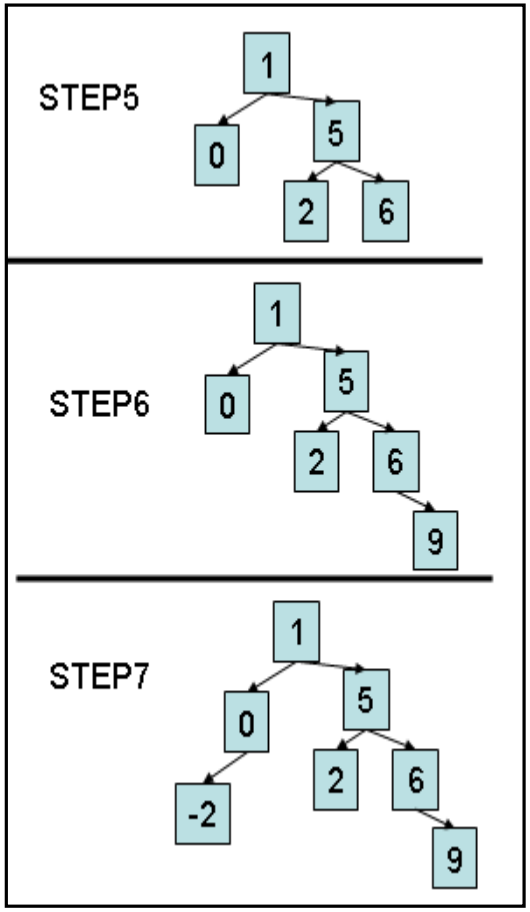
```
main()
{
     BTREE myroot =NULL;
     int i=0;

     scanf("%d",&i);
     while(i!=-1)
     {
       myroot=insert(myroot,i);
       scanf("%d",&i);
     }

}
```

// INPUT VALUES 1 5 6 2 0 9 -2





BINARY TREE TRAVERSALS 1/5

- Several ways to visit nodes(elements) of a tree

Inorder	**Preorder**	**Postorder**
1. Left subtree	1. Root	1. Left subtree
2. Root	2. Left subtree	2. Right subtree
3. Right subtree	3. Right subtree	3. Root

BINARY TREE TRAVERSALS 2/5

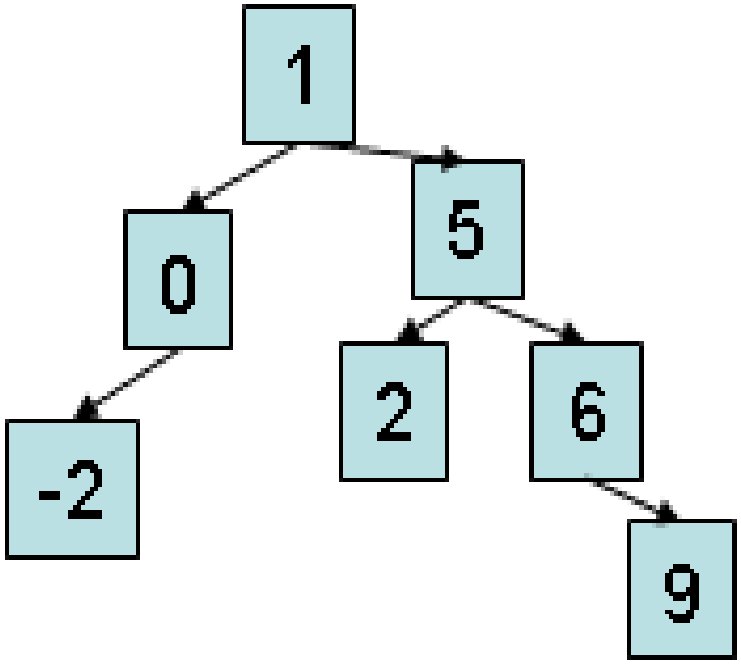
```
void inorder(BTREE root)
{    if(root!=NULL)
      {  inorder(root->left);
         printf("%d",root->data);
         inorder(root->right);       }
}
```

```
void preorder(BTREE root)
{
 if(root!=NULL)
      {  printf("%d",root->data);
         preorder(root->left);
         preorder(root->right);       }
}
```

```
void postorder(BTREE root)
{    if(root!=NULL)
      {  postorder(root->left);
         postorder(root->right);
         printf("%d",root->data);  }
}
```

BINARY TREE TRAVERSALS 3/5

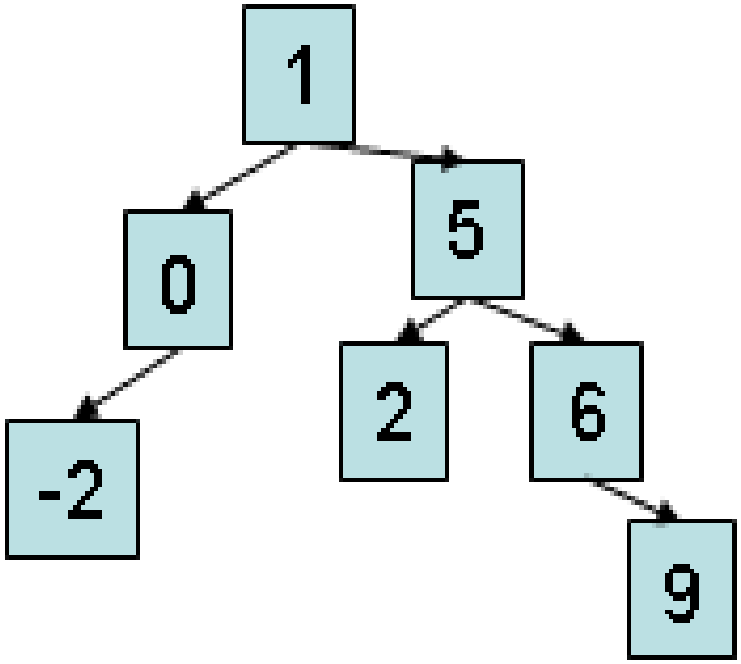
```
void inorder(BTREE root)
{    if(root!=NULL)
     {  inorder(root->left);
        printf("%d",root->data);
        inorder(root->right);
}
```



// OUTPUT : -2 0 1 2 5 6 9

BINARY TREE TRAVERSALS 4/5

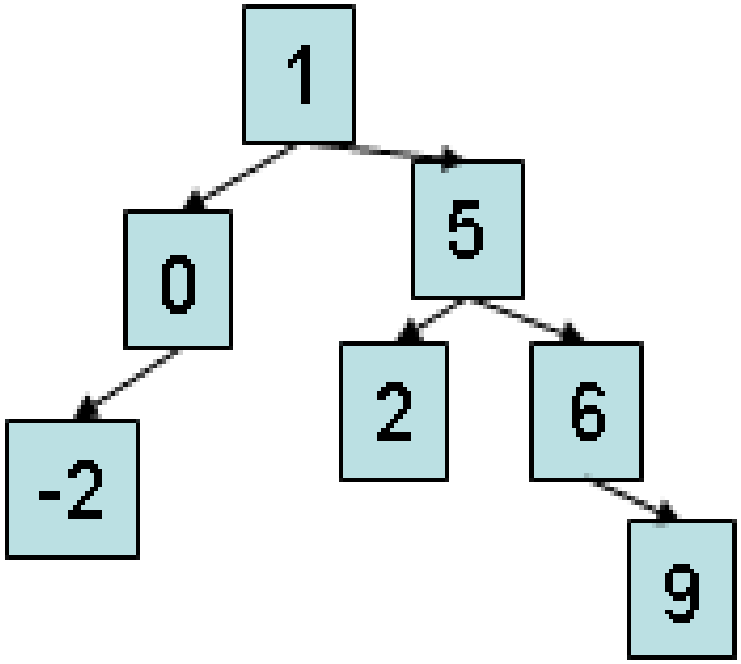
```
void preorder(BTREE root)
{
 if(root!=NULL)
      {   printf("%d",root->data);
          preorder(root->left);
          preorder(root->right);        }
}
```



// OUTPUT : 1 0 -2 5 2 6 9

BINARY TREE TRAVERSALS 5/5

```
void postorder(BTREE root)
{    if(root!=NULL)
      {   postorder(root->left);
          postorder(root->right);
          printf("%d",root->data);  }
}
```



// OUTPUT : -2 0 2 9 6 5 1

FIND SIZE OF A TREE

```
int size ( BTREE root)
{
    if(root!=NULL)
        return(size(root->left) + 1 + size(root->right));
    else return 0;
}
```

Max Depth of a tree

```
int maxDepth(BTREE node)
{   int lDepth;        int rDepth;

    if (node==NULL)          return(0);

  else
{    // compute the depth of each subtree
    lDepth = maxDepth(node->left);
    rDepth = maxDepth(node->right);

    // use the larger one
    if (lDepth > rDepth)     return(lDepth+1);
    else                     return(rDepth+1);
  }
}
```

Delete a node from a tree (1/5)

BTREE delete_node(BTREE root,int x) // SEARCH AND DELETE x in tree

1. x> root->data → search right subtree
2. x<root->data → search left subtree
3. root->data==x

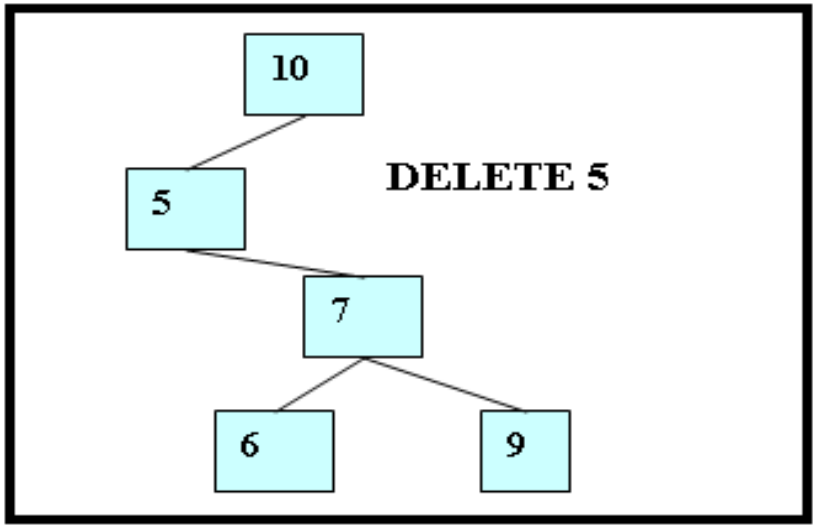
   3.1 root is a leaf node → free root =free tree
   3.2 root has no left subtree → root=root->right
   3.3 root has no right subtree → root=root->left
   3.4 root has right and left subtree ☹ → append right subtree to left subtree

Delete a node from a tree (2/5)

BTREE delete_node(BTREE root,int x) // SEARCH AND DELETE x in tree

1. x> root->data → search right subtree
2. x<root->data → search left subtree
3. root->data==x

   3.1 root is a leaf node
   3.2 root has no left subtree → root=root->right
   3.3 root has no right subtree
   3.4 root has right and left subtree ☹



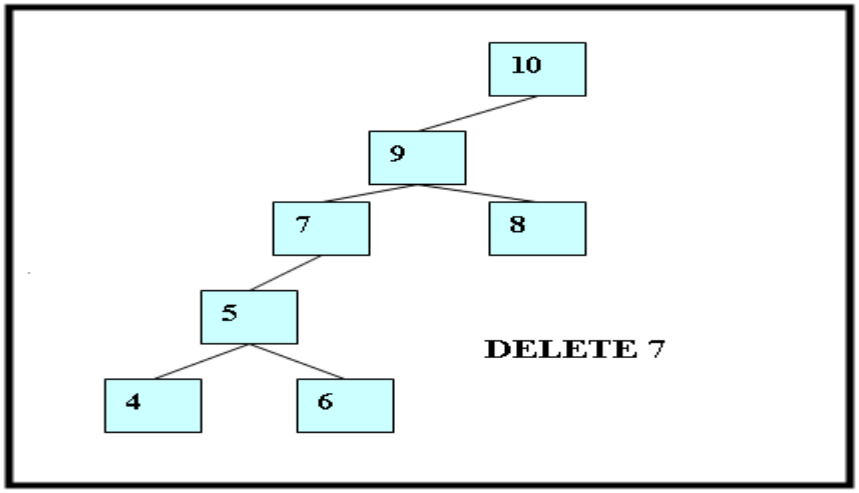
3.2nd Case)

Delete a node from a tree (3/5)

BTREE delete_node(BTREE root,int x) // SEARCH AND DELETE x in tree

1. x> root->data → search right subtree
2. x<root->data → search left subtree
3. root->data==x

   3.1 root is a leaf node
   3.2 root has no left subtree
   3.3 root has no right subtree → root=root->left
   3.4 root has right and left subtree ☹



(3. 3th Case)

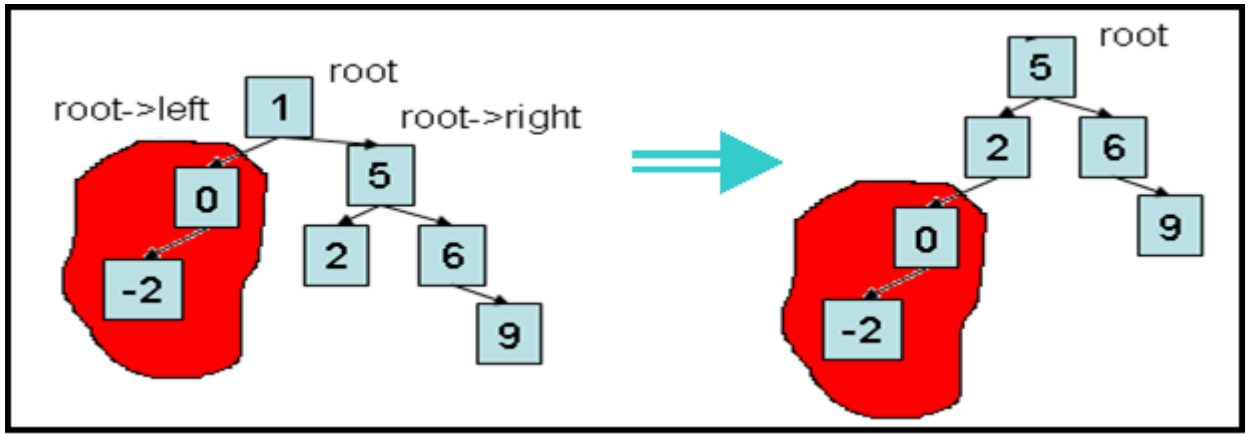
Delete a node from a tree (4/5)

BTREE delete_node(BTREE root,int x) // SEARCH AND DELETE x in tree

1. x> root->data → search right subtree
2. x<root->data → search left subtree
3. root->data==x

   3.1 root is a leaf node
   3.2 root has no left subtree
   3.3 root has no right subtree
   3.4 root has right and left subtree ☹ → append right subtree to left subtree

(3.4th Case)



Delete a node from a tree (5/5)

```
BTREE delete_node(BTREE root,int x)
{    BTREE p,q;

     if(root==NULL) return NULL; // no tree

     if(root->data==x)                          // find x in root
     {       if(root->left==root->right)        // root is a  leaf node
                        { free(root); return NULL; }
             else       {
                         if(root->left==NULL)            { p=root->right; free(root); return p; }
                         else if(root->right==NULL)      { p=root->left; free(root); return p;   }
                         else {      p=q=root->right;
                                     while(p->left!=NULL) p=p->left;
                                     p->left=root->left;
                                     free(root);
                                     return q;   }
                        }
     }
     if(root->data<x)                { root->right=delete_node(root->right,x); }
     else                            { root->left=delete_node(root->left,x); }
     return root;

}
```

Search a node in a tree

- Search in binary trees requires O(log *n*) time in the average case, but needs O(*n*) time in the worst-case, when the unbalanced tree resembles a linked list

- **PSEUDOCODE**

```
search_binary_tree(node, key)
{       if ( node is NULL)  return None // key not found
        if (key < node->key)
                                return search_binary_tree(node->left, key)
        elseif (key > node->key)
                                return search_binary_tree(node->right, key)
        else                    return node // found key
}
```